BSc Software Engineering Group B Morning Group 6 Project Work

Week 1 Threat Model Document

Project 6: Zero-Knowledge Password Manager

Name	Index
Stedan Yaw Boah Ampofo	1696596666
Prince Lateef Abdul Rahman	1703675677
Keith Yaotey	1694883181
Prince Opoku Owusu	1702131132
Sedoame Kofi Ahorlu	1704464006
Bright Kportiklah	1693948139
Cyril Prince Agbeve	1701864653

This document establishes the formal security architecture, cryptographic trust boundaries, level-1 data flow representations, and comprehensive STRIDE threat register for Project 6: Zero-Knowledge Password Manager. The system enforces a mathematically verifiable zero-knowledge paradigm: all cryptographic key derivation, authenticated symmetric encryption, asymmetric key agreement, and zero-knowledge authentication handshakes execute exclusively inside the browser extension client runtime. The central Express application server functions strictly as an untrusted blind relay, and the underlying PostgreSQL persistence tier retains ciphertext exclusively.

Course Project	Project 6: Zero-Knowledge Password Manager
Milestone	Week 1 Threat Model Document (Baseline Submission)
Core Cryptography	Argon2id (KDF), XChaCha20-Poly1305 (AEAD), OPAQUE (aPAKE), X25519 (ECDH)
System Architecture	Browser Extension (Client Core) • Express.js (Blind Relay) • PostgreSQL (Ciphertext Only)
Trust Boundaries	TB-1 (Client), TB-2 (Application Server), TB-3 (Database), TB-4 (External API - HIBP)
Threat Tooling	OWASP Threat Dragon (docs/threat-model.json) • STRIDE-per-Element

1. Introduction

1.1 Purpose

This document establishes the foundational Week 1 Threat Model Document for Project 6: Zero-Knowledge Password Manager. Centralized credential managers represent high-value targets for adversaries ranging from opportunistic credential harvesters to advanced persistent threats (APTs). A single architectural flaw, improper key handling routine, or memory leak can lead to catastrophic compromise across an end user's entire digital presence.

The primary objective of this threat model is to systematically identify, categorize, and establish mitigations for security risks at the earliest phase of the software development lifecycle (SDLC). By conducting a structured STRIDE-per-Element analysis prior to implementation, Group 6 enforces a rigorous security-by-design methodology. This guarantees that the system upholds its core mathematical pledge: absolute zero-knowledge privacy where neither the host server nor the underlying persistence database ever accesses unencrypted credentials, master keys, or derivation secrets.

1.3 Scope

To ensure clear analytical focus, the threat modeling boundary explicitly delineates in-scope subsystems and out-of-scope operational risks.

In-Scope Components:

- **Browser Extension Runtime:** Chromium/Firefox extension built under Manifest V3, including popup views, background service workers, isolated content scripts, and secure browser storage APIs.
- **Cryptographic Primitives:** Client-side execution of Argon2id password-based key derivation, XChaCha20-Poly1305 authenticated encryption with associated data (AEAD), OPAQUE asymmetric password-authenticated key exchange (aPAKE), and X25519 elliptic-curve Diffie-Hellman key sharing.
- **Application Relay Server:** Node.js/Express service acting as a blind relay for opaque ciphertext payloads, authentication coordination, and public key directory synchronization.
- **Database Persistence Tier:** PostgreSQL relational database storing user records, blinded authentication credentials, encrypted vault blobs, and audit logs.
- **Third-Party Integrations:** Outbound Have I Been Pwned (HIBP) API communications utilizing 5-character prefix k-anonymity.
- **Core Workflows:** Vault registration, zero-knowledge authentication, continuous multi-device synchronization, peer-to-peer item sharing, and timelocked emergency access recovery.

Out-of-Scope Components:

- **Host Endpoint Compromise:** Physical device seizure, kernel-level keyloggers, rootkits, or compromised hypervisors operating beneath the browser execution sandbox.
- **Public CA Infrastructure:** Compromise or rogue certificate issuance by public Certificate Authorities within the TLS trust tree.
- **Infrastructure-Level Volumetric DDoS:** Multi-gigabit network-layer distributed denial-of-service attacks against hosting data centers, managed by upstream cloud edge mitigation providers.
- **Human Social Engineering:** Direct physical coercion, bribery, or out-of-band social engineering targeting vault owners.

2. System Overview

2.1 Architecture Summary

Project 6 enforces a strict Zero-Knowledge Architecture. The application server and persistence tiers operate as untrusted conduits, possessing zero access to plaintext passwords, encryption keys, or decrypted vault records. All sensitive operations are client-centric:

- **Argon2id (Password Hashing & Key Derivation):** Argon2id executes exclusively within the client extension to derive the Master Encryption Key (MEK) and OPAQUE authentication secrets. Configured with memory-hard parameters (iterations=3, memory=64MB, parallelism=4), Argon2id provides robust defense against GPU/ASIC-accelerated brute-force attacks and side channels.
- **XChaCha20-Poly1305 (AEAD Vault Encryption):** Vault records are encrypted using XChaCha20-Poly1305 with an extended 192-bit nonce and 256-bit symmetric keys. The 192-bit nonce eliminates the risk of nonce reuse during asynchronous multi-device synchronization, while Poly1305 validates ciphertext integrity before decryption.
- **OPAQUE Protocol (Zero-Knowledge Authentication):** An asymmetric password-authenticated key exchange (aPAKE) protocol that allows mutual authentication without transmitting the master password or storing password hashes on the server that could be cracked offline if the database is leaked.
- **X25519 (Public-Key Cryptography for Sharing):** Secure item sharing between vault owners is facilitated via X25519 Diffie-Hellman key agreement. Ephemeral shared secrets encrypt individual item keys, allowing cryptographic delegation without exposing master keys.
- **Express Application Server (Blind Relay):** The server inspects request routing envelopes, validates session tokens, enforces rate limits, and relays ciphertext payloads to PostgreSQL without holding decryption capabilities.
- **PostgreSQL Database (Ciphertext Storage):** Persists opaque encrypted blobs, blinded credential salts, version numbers, and recovery state tables.

2.2 Actors and Assets

The system models interactions between five core actors operating across critical organizational and personal assets.

System Actors:

- **Legitimate Vault Owner:** Authenticated end user managing credentials via the client browser extension.
- **Authorized Recipient / Emergency Contact:** Registered peer user receiving shared credentials or granted timelocked emergency vault recovery access.
- **Network Adversary (Man-in-the-Middle):** Hostile entity observing, intercepting, or altering network traffic over public transit paths.
- **Malicious Web Origin / Injected Script:** Hostile web applications attempting DOM-based credential theft, prototype pollution, or clickjacking.
- **Compromised Server Operator / Rogue Admin:** Malicious insider or adversary possessing root privileges on Express or PostgreSQL instances.

System Assets:

- **Master Password & Key Material:** Ephemeral cleartext password (strictly in volatile client RAM) and derived Master Encryption Key (MEK).
- **Vault Ciphertext & Item Keys:** Encrypted application credentials, notes, TOTP seeds, and associated Poly1305 authentication tags.
- **Asymmetric Private Keys:** Client-generated X25519 private keys stored encrypted within the client vault.
- **OPAQUE Registration Records:** Server-stored blinded credential records enabling zero-knowledge authentication.
- **Integrity Metadata & Timelocks:** Monotonic vault version counters, emergency access timelocks, and append-only audit event logs.

2.3 Trust Boundaries

The architecture partitions components across four explicit Trust Boundaries (TB-1 through TB-4):

- **TB-1: Client Trust Zone (User + Browser Extension):** The primary trusted boundary. Encapsulates user input, the extension popup UI, background service workers, and volatile memory holding decrypted secrets. Assumes the browser security sandbox is uncompromised.
- **TB-2: Application Server Trust Zone (Express Server):** The untrusted network boundary. Handles incoming HTTP requests, enforces rate limits, validates payload schemas, and

routes blinded cryptographic tokens. The server is untrusted with regard to vault confidentiality.

- **TB-3: Database Trust Zone (PostgreSQL):** The storage boundary. Persists encrypted blobs, blinded salts, and metadata. Fully untrusted: complete database exfiltration reveals zero plaintext credentials.
- **TB-4: External API Boundary (Have I Been Pwned API):** The external boundary governing outbound password breach verification over TLS 1.3, mediated exclusively through a 5-character prefix k-anonymity protocol.

3. Level-1 Data Flow Diagram

3.1 DFD Description

The system's data flows are modeled across twelve distinct pathways (DF-1 through DF-12) traversing the four trust boundaries:

- **DF-1: Master Password Ingestion & Key Derivation (User → Browser Extension):** User inputs master password into the extension UI. The extension immediately executes client-side Argon2id to derive the Master Encryption Key (MEK) and OPAQUE credential material. Decrypted keys remain strictly in volatile memory.
- **DF-2: OPAQUE User Registration Flow (Browser Extension ↔ Express Server):** The client and Express server conduct the OPAQUE registration handshake over TLS 1.3, allowing the server to store a blinded credential record without learning the password or master key.
- **DF-3: OPAQUE Authentication Exchange (Browser Extension ↔ Express Server):** Upon login, client and server execute the multi-round OPAQUE aPAKE exchange over TLS 1.3, mutually authenticating and establishing an ephemeral session token.
- **DF-4: Encrypted Vault Synchronization Push (Browser Extension → Express Server):** Client encrypts updated vault records locally with XChaCha20-Poly1305 and transmits the ciphertext payload alongside a monotonic version counter to the Express relay.
- **DF-5: Vault Ciphertext Retrieval & Pull (Express Server → Browser Extension):** Client requests latest vault state. Express server retrieves and relays ciphertext to the client, which validates version monotonicity and decrypts records locally.
- **DF-6: X25519 Public Key Registration & Directory Distribution (Browser Extension ↔ Express Server):** Client generates an X25519 keypair, publishes the public key to the Express directory service, and stores the encrypted private key in the vault.
- **DF-7: Secure Item Sharing Handshake & Authorization (Sender Extension ↔ Express Server ↔ Recipient):** Sender requests recipient's authenticated public key from the directory, initiating an asymmetric key agreement handshake.

- **DF-8: Encrypted Shared Secret Payload Transmission (Sender Extension → Express Server → Recipient):** Sender encrypts the shared item key using ephemeral X25519 ECDH and transmits the ciphertext envelope through Express to the recipient.
- **DF-9: Ciphertext Persistence & Query Execution (Express Server ↔ PostgreSQL Database):** Express server executes parameterized SQL queries over an internal TLS channel to store or retrieve encrypted blobs and blinded user records in PostgreSQL.
- **DF-10: Compromised Password Auditing via K-Anonymity (Browser Extension → HIBP API):** Extension hashes a password with SHA-1, truncates it to a 5-character prefix, queries HIBP API, and performs local suffix comparison against returned breached candidates.
- **DF-11: TOTP Seed Storage & Local Code Generation (Browser Extension Vault ↔ Service Worker):** Extension decrypts stored TOTP seeds locally in memory using the item key, generates time-based one-time codes, and renders them in the UI.
- **DF-12: Emergency Access Request & Timelocked Recovery (Browser Extension ↔ Express Server ↔ Recovery Contact):** Designated contact initiates recovery. Express server starts a tamper-evident waiting period (timelock). Upon expiration without owner veto, the pre-shared recovery key is released.

3.2 Diagram

Figure 3.1 depicts the Level-1 Data Flow Diagram across Trust Boundaries TB-1 through TB-4 and Data Flows DF-1 through DF-12.

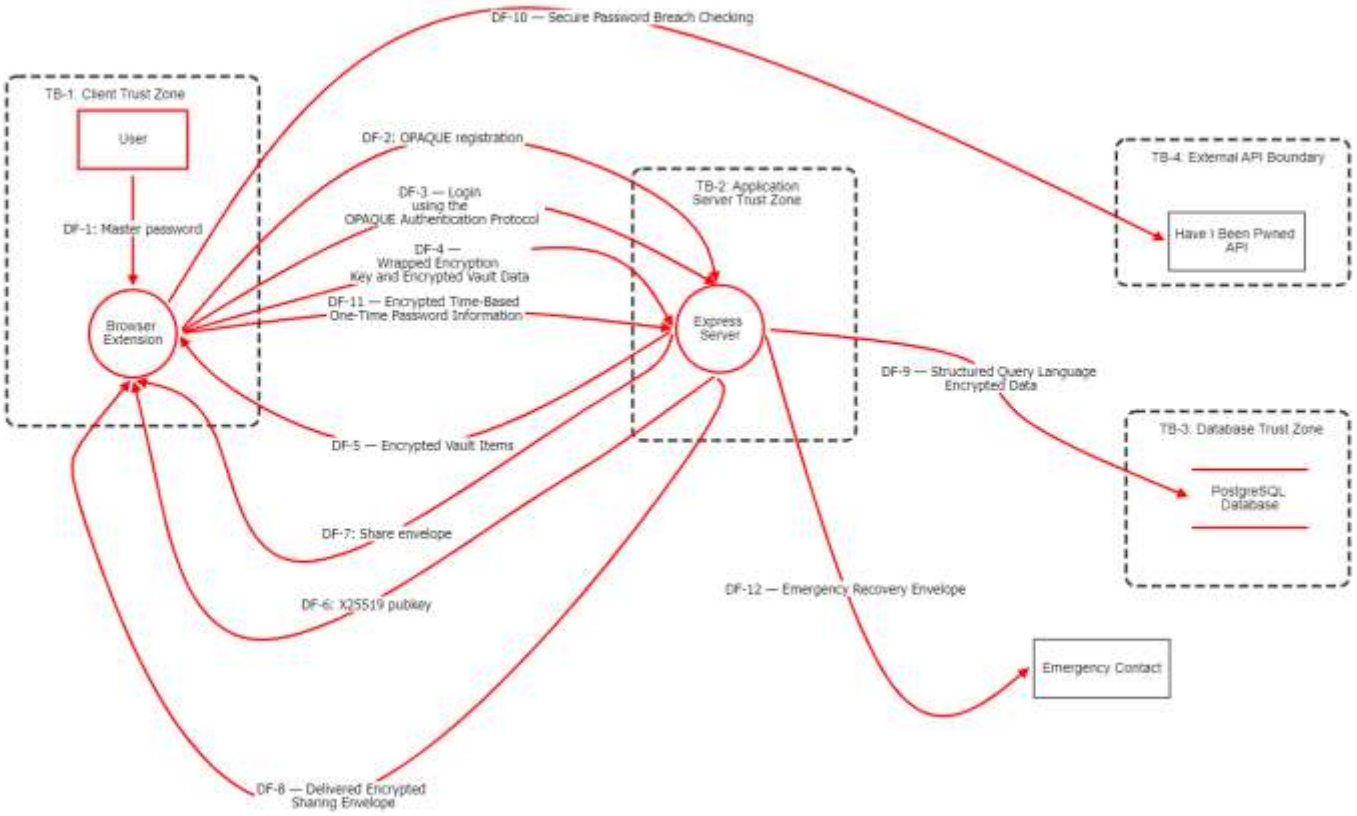


Figure 3.1: Level-1 Data Flow Diagram for Project 6: Zero-Knowledge Password Manager. Delineates data flows DF-1 to DF-12 across Client Trust Zone (TB-1), Application Server (TB-2), Database (TB-3), and External HIBP API (TB-4).

4. STRIDE-per-Element Analysis

4.1 Methodology

The security posture of Project 6 is systematically assessed using the STRIDE threat modeling framework. Each architectural element and data flow is evaluated across six threat categories: Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privilege. Risk severities (Critical, High, Medium, Low) are assigned based on exploitability, impact on zero-knowledge confidentiality, and potential blast radius.

4.2 Threat Register

Table 4.1 details all twenty-one (21) identified architectural threats across the system's operational elements.

ID	Element	STRIDE Category	Threat Description	Severity	Mitigation
T-01	DF-3	Spoofing	MitM on OPAQUE login attempting credential interception.	Medium	TLS 1.3 + OPAQUE mutual authentication.
T-02	DF-6	Spoofing	X25519 public key impersonation in sharing directory.	High	Cryptographic signatures validating public key ownership.
T-03	DF-9	Tampering	Bit flip or payload modification in stored vault ciphertext.	Critical	XChaCha20-Poly1305 AEAD integrity check and tag.
T-04	DF-12	Tampering	Emergency recovery timer manipulation to force early access.	Medium	Signed timestamps with server consensus clock validation.
T-05	DF-7	Repudiation	Denial of password sharing transaction by sender.	Low	Client-signed transactions with asymmetric keypairs.
T-06	DF-12	Repudiation	Denial of emergency recovery invocation by contact.	Low	Tamper-evident, append-only server audit logs.
T-07	DF-1	Info Disclosure	Master password theft via malicious DOM script injection.	Critical	Isolated MV3 contexts + strict Content Security Policy.
T-08	DF-9	Info Disclosure	Database exfiltration and application server log theft.	Critical	Zero-knowledge architecture; server holds only ciphertext.
T-09	DF-10	Info Disclosure	HIBP hash disclosure leaking candidate password hashes.	Critical	5-character SHA-1 prefix k-anonymity protocol.

T-10	DF-11	Info Disclosure	TOTP seed leakage during code generation or storage.	Medium	Encrypt TOTP seed with item_key in encrypted record.
T-11	DF-3	Denial of Service	Authentication service flooding via automated requests.	High	IP rate limiting + progressive backoff challenges.
T-12	DF-4	Denial of Service	Sync API flooding with excessive payload sizes or bursts.	High	Request size limits (max 5MB) + token bucket rate limits.
T-13	DF-12	Elevation of Privilege	Emergency recovery timer bypass to gain premature access.	High	Atomic state transitions in PostgreSQL via triggers.
T-14	Browser Ext	Elevation of Privilege	Phishing via browser frame impersonation or spoofed tabs.	Critical	Exact-origin matching via webNavigation API.
T-15	DF-9	Elevation of Privilege	IDOR leading to cross-tenant vault record access or write.	High	Parameterized SQL queries binding authenticated user_id.
T-16	DF-8	Tampering	Modification of encrypted sharing envelope in transit.	Medium	AEAD encapsulation of shared secrets with associated data.
T-17	DF-2	Spoofing	OPAQUE registration replay attack targeting account setup.	Medium	Server-side replay protection using ephemeral nonces.
T-18	DF-5	Tampering	Vault rollback attack replacing latest vault	High	Monotonic version numbers bound

			with stale blob.		within AEAD AAD.
T-20	User	Elevation of Privilege	Weak master password vulnerable to client-side brute forcing.	High	Client-side zxcvbn strength meter + entropy enforcement.
T-21	Express Server	Elevation of Privilege	Server compromise via vulnerable npm third-party dependency.	High	Automated npm audit in CI/CD + non-root container user.
T-22	PostgreSQL	Denial of Service	Database storage exhaustion via runaway sync payloads.	Medium	Per-user storage quotas + automated storage monitoring.

4.3 Risk Rating Distribution

Table 4.2 summarizes the distribution of the twenty-one cataloged threats by severity level.

Severity Rating	Threat Count	Percentage of Total	Architectural Impact Summary
Critical	5	23.8%	Existential risks: DOM injection, Ciphertext tampering, DB theft, HIBP leak, Phishing
High	8	38.1%	Significant risks: Key spoofing, DoS flooding, Rollback, IDOR, Weak passwords, npm CVEs
Medium	6	28.6%	Operational risks: Registration replay, MitM sniffing, TOTP leak, Storage quotas, Timers
Low	2	9.5%	Audit & Non-repudiation: Sharing transaction denial, Emergency recovery denial
Total	21	100.0%	Complete System Attack Surface Coverage

The risk distribution confirms that 61.9% of all cataloged threats fall into the Critical (23.8%) or High (38.1%) categories. Because the server is mathematically blind, core security guarantees reside at the client perimeter (DOM security, anti-phishing) and within cryptographic encapsulation (AEAD integrity, k-anonymity), while server controls focus on availability, rate limiting, and access isolation.

5. Top Five Threats and Controls

The five threats classified as Critical (T-07, T-03, T-08, T-09, and T-14) present existential risks to the zero-knowledge security guarantees of Project 6. An unmitigated exploit in any of these five

vectors directly leads to cleartext credential leakage, unauthorized vault decryption, or cryptographic malleability. Detailed analyses and controls are provided below.

5.1 Threat T-07: Master Password Theft via DOM Injection (DF-1)

Threat Mechanics & Attack Vector: In browser extensions, content scripts execute in the context of host web pages. If extension UI elements (e.g., autofill dropdowns, in-page prompts, or improperly isolated iframes) interact directly with host DOM structures, malicious web scripts can exploit Cross-Site Scripting (XSS) or DOM prototype pollution. An attacker can hook input event listeners or inspect volatile DOM trees to capture the master password in cleartext as the user types.

Impact Analysis: Total catastrophic compromise. Exfiltration of the cleartext master password allows an adversary to run Argon2id locally and decrypt the entire vault offline once ciphertext is obtained.

- **Isolated Manifest V3 Contexts:** Master password entry is strictly confined to extension action popups and dedicated options pages running in isolated browser processes inaccessible to web page JavaScript.
- **Strict Content Security Policy (CSP):** The extension enforces default-src 'none'; script-src 'self'; style-src 'self'; frame-ancestors 'none';, completely disabling inline scripts, remote script loading, and frame embedding.
- **Volatile Memory Sanitization:** Master passwords are held in Uint8Array buffers that are actively zeroized immediately following key derivation.

5.2 Threat T-03: Bit Flip & Tampering in Stored Vault Ciphertext (DF-9)

Threat Mechanics & Attack Vector: An adversary with database access or storage-layer interception capabilities can flip bits within stored ciphertext blobs. In unauthenticated cipher constructions (e.g., standard AES-CBC), bit flipping allows predictable plaintext alterations or padding oracle exploits.

Impact Analysis: Undetected vault corruption, record redirection, or cryptographic malleability compromises.

- **XChaCha20-Poly1305 AEAD Integrity Checks:** Encryption utilizes authenticated encryption with associated data. The 128-bit Poly1305 authenticator tag validates ciphertext integrity: any modified bit causes decryption to immediately fail, rejecting the payload prior to parsing.
- **HMAC-SHA256 Manifest Digest:** A root cryptographic checksum binds all vault records, ensuring that partial record deletions or substitutions in PostgreSQL are detected during client synchronization.

5.3 Threat T-08: Database Exfiltration & Application Server Log Theft (DF-9)

Threat Mechanics & Attack Vector: Adversaries breach backend servers through SQL injection, exposed backup archives, or host privilege escalation. In traditional architectures, this leaks password hashes susceptible to offline cracking.

Impact Analysis: In a conventional system, total database compromise results in mass credential harvesting.

- **Enforced Zero-Knowledge Architecture:** The Express relay and PostgreSQL database store ciphertext exclusively. The server never receives, generates, or stores master passwords or derived keys.
- **Memory-Hard Key Derivation:** Vault keys are derived using client-side Argon2id (iterations=3, memory=64MB, parallelism=4). Stolen ciphertext is computationally intractable to decrypt offline.
- **Zero-Log Relay Policy:** Express middleware actively scrubs authorization headers and encrypted payload bodies from all operational logs.

5.4 Threat T-09: HIBP Hash Disclosure During Breach Audits (DF-10)

Threat Mechanics & Attack Vector: Transmitting full password hashes or plaintext credentials to third-party breach monitoring APIs exposes user passwords to external network eavesdroppers or third-party breaches.

Impact Analysis: Direct loss of credential confidentiality and deanonymization of sensitive passwords.

- **5-Character Prefix K-Anonymity:** The extension hashes candidate passwords locally with SHA-1 and sends only the first 5 hexadecimal characters (20 bits) to the HIBP API over TLS 1.3.
- **Local Suffix Matching:** The API returns all known breached hash suffixes matching that prefix. The extension evaluates the remaining 35 characters entirely in local client memory. The external API never learns the full hash or matching status.

5.5 Threat T-14: Phishing via Browser Frame Impersonation (Browser Extension)

Threat Mechanics & Attack Vector: Malicious websites use deceptive homograph domains, hidden cross-origin iframes, or spoofed login popups to trick the browser extension into autofilling credentials into attacker-controlled form fields.

Impact Analysis: Direct credential theft and unauthorized account takeover on targeted third-party web platforms.

- **Exact-Origin Matching:** The extension binds credential matching strictly to the fully qualified domain name (FQDN), HTTPS protocol, and port of the top-level browsing context via the webNavigation API. Autofill is completely disabled within cross-origin or sandboxed iframes.
- **Mandatory User Gesture:** Autofill requires an explicit user action (e.g., clicking the extension icon); passive automatic injection on page load is strictly prohibited.
- **Visual Domain Indicators:** The extension UI displays the verified top-level domain prominently before prompting for autofill authorization.

6. Threat Model Maintenance

6.1 Iterative Maintenance Strategy & Roadmap

Threat modeling is an active, iterative engineering discipline integrated into the development lifecycle. This model will undergo two mandatory formal synchronization reviews:

- **Week 4 Milestone Review (Architecture & Prototype Alignment):** Scheduled upon prototype completion. Group 6 will validate implemented cryptographic primitives (OPAQUE handshake, Argon2id parameters) against Table 4.1 mitigations, verify client memory sanitization, and assess emerging API schemas and dependencies.
- **Week 10 Milestone Review (Pre-Production Hardening & Verification):** Scheduled prior to final system release. Integrates findings from penetration testing, dynamic API fuzzing, and automated static security scans (SAST). Confirms automated test suites for all Critical and High threats operate reliably under adversarial scenarios.

6.2 Tooling Integration & Version Control

The threat model is maintained using the open-source OWASP Threat Dragon modeling suite. The Threat Dragon JSON file is located at docs/threat-model.json.

The JSON model is version-controlled directly within the primary project repository. Any architectural modifications require corresponding updates to docs/threat-model.json and peer approval via pull request reviews. The CI/CD pipeline enforces continuous security checks:

- **Dependency Auditing:** Automated execution of npm audit --audit-level=high and container scanning to mitigate T-21.
- **Static Security Linting:** Automated rules enforcing parameterized queries (T-15) and blocking DOM-based XSS vectors (T-07).
- **Cryptographic Test Vectors:** Unit test execution against verified RFC test vectors for Argon2id, XChaCha20-Poly1305, and X25519.

6.3 Engineering Conclusion & Sign-Off

The Week 1 Threat Model Document establishes a rigorous security foundation for Project 6: Zero-Knowledge Password Manager. By enforcing client-side cryptographic isolation, zero-

knowledge server boundaries, and multi-layered controls across all twenty-one STRIDE threats, Group 6 guarantees verifiable privacy and resilience against advanced adversarial threats.